JC2 语言规范

第一卷：源文本、文法与求值语义

中文草案

版本 0.1

2026 年 11 月

本卷规定 JC2 源文本的词法结构、抽象文法、名字与类型系统、表达式求值、声明与模块语义，以及一致性实现应当满足的可观察行为。

目录

1 总则 **5**
1.1 范围 . . . 5
1.2 规范性用语 . . . 5
1.3 一致性 . . . 5
1.4 版本 . . . 6
1.5 统一表达式模型 . . . 6

2 源文本 **6**
2.1 编码 . . . 6
2.2 行与空白 . . . 6
2.3 注释 . . . 7
2.4 词法单元 . . . 7
2.5 标识符与关键字 . . . 7
2.6 数字字面量 . . . 8
2.7 字符串字面量 . . . 8
2.8 运算符与分隔符 . . . 9
2.9 词法文法 . . . 9
2.10 编译器指令 . . . 10
2.11 词法错误与位置 . . . 11

3 文法 **11**
3.1 记法 . . . 11
3.2 程序与表达式序列 . . . 12
3.3 声明表达式 . . . 12
3.4 控制表达式 . . . 13
3.5 表达式 . . . 14
3.6 字面量与构造 . . . 15
3.7 参数与实参 . . . 15
3.8 模式 . . . 16
3.9 句法歧义消解 . . . 17
3.9.1 dict 字面量与块的区分 . . . 17
3.10 文法的静态约束 . . . 18

4 名字、绑定与作用域 **18**
4.1 绑定 . . . 18
4.1.1 重声明与遮蔽 . . . 19

4.1.2 局部捕获与闭包 . . . 19
4.2 作用域 . . . 20
4.2.1 解析顺序 . . . 20
4.3 提升 . . . 20
4.4 修饰符 . . . 21
4.4.1 ref 绑定 . . . 21
4.4.2 state 绑定 . . . 21
4.4.3 const 与删除 . . . 21
4.5 模块与 include . . . 22

5 类型与值 **22**
5.1 值域 . . . 22
5.2 类型 . . . 22
5.2.1 类型对象与驻留 . . . 23
5.2.2 原子类型与联合类型 . . . 23
5.2.3 类型注解与 as 断言 . . . 23
5.2.4 类型对象调用 . . . 24
5.2.5 行为谓词 . . . 24
5.3 真值 . . . 25
5.4 数值 . . . 25
5.5 相等、同一性与排序 . . . 25
5.5.1 哈希规则 . . . 26
5.6 复制与冻结 . . . 26

6 表达式求值 **27**
6.1 求值关系 . . . 27
6.2 基本规则 . . . 27
6.3 求值顺序 . . . 27
6.4 优先级与结合性 . . . 28
6.5 算术 . . . 29
6.6 比较与逻辑 . . . 29
6.7 调用 . . . 29
6.8 赋值 . . . 30
6.9 索引与切片 . . . 30

7 控制表达式 **30**
7.1 表达式的值 . . . 30
7.2 块 . . . 31

7.3 条件与循环 . . . 31
7.4 for-in . . . 32
7.5 match . . . 32
7.6 defer . . . 32
7.7 try、catch 与 throw . . . 32

8 函数与参数 33
8.1 闭包 . . . 33
8.1.1 参数绑定算法 . . . 33
8.2 默认参数 . . . 34
8.3 ref 与 const 参数 . . . 34
8.4 rest、kwargs 与 spread . . . 34
8.5 占位符与部分应用 . . . 35

9 模式与解构 35
9.1 模式种类 . . . 35
9.2 匹配 . . . 35
9.2.1 分支选择与 guard . . . 36
9.2.2 动态断言与求值顺序 . . . 36
9.2.3 容器模式 . . . 36
9.2.4 视图提取 . . . 37
9.3 解构赋值 . . . 37
9.4 state 目标 . . . 37

10 声明表达式 37
10.1 变量声明 . . . 37
10.2 class 与 trait . . . 37
10.3 属性 . . . 38
10.4 属性查找 . . . 38
10.5 类型一致性 . . . 39
10.6 继承与初始化 . . . 39
10.7 宿主类引用 . . . 40
10.8 trait 组合 . . . 40
10.9 dunder 协议 . . . 40
10.10 反射 . . . 41
10.11 enum . . . 42

11 namespace 与模块 **42**
11.1 namespace . . . 42
11.2 include . . . 43
11.3 导入 . . . 43

12 宏与 syntax **44**
12.1 macro . . . 44
12.2 宏卫生 . . . 45
12.3 ASTNode 结构 . . . 46
12.4 syntax . . . 47

13 内建值域 **48**
13.1 字符串 . . . 48
13.2 容器 . . . 48
13.2.1 list . . . 49
13.2.2 dict . . . 49
13.2.3 set . . . 49
13.2.4 哈希与键 . . . 49
13.3 矩阵 . . . 49

14 错误 **50**
14.1 异常值 . . . 50
14.2 内置错误类型 . . . 50
14.3 错误传播 . . . 51
14.4 assert . . . 52

15 一致性要求 **52**

A 文法汇总 **52**
A.1 词法产生式 . . . 52
A.2 程序与表达式 . . . 54
A.3 构造与字面量 . . . 55
A.4 声明与成员 . . . 56
A.5 控制表达式 . . . 57
A.6 参数、实参与推导式 . . . 57
A.7 模式 . . . 58

B 一致性矩阵 **58**

C 规范用语索引 **59**

1 总则

1.1 范围

本规范规定 JC2 源语言的以下部分：

1. 源文本的编码、词法单元与字面量；
2. 程序、表达式、控制形式与模式的文法；
3. 名字、绑定、作用域与声明的静态语义；
4. 值、类型、表达式求值与控制流的动态语义；
5. 函数、类、trait、namespace、模块与宏的语义；
6. 错误类型、错误传播与一致性要求。

本规范以源语言的可观察行为为对象。实现可以选择解释执行、字节码执行或即时编译；这些选择属于实现结构，规范以程序的可观察结果定义语义。第一卷定义语言核心值域。

1.2 规范性用语

本规范使用以下用语：

用语	含义
“必须”	一致性实现应当满足的绝对要求
“应当”	一致性实现应当满足的要求，例外需要完整记录
“可以”	实现或程序可以选择的自由
“产生错误”	求值或静态检查以指定错误结束
“保持”	状态在指定步骤之后维持该性质

条文中的“错误”指第 14 节定义的异常值。“程序”指一个源文件及其导入闭包组成的编译单元集合。

1.3 一致性

一致性实现必须满足：

1. 接受本规范文法规定的全部合式程序；
2. 对合式程序产生本规范规定的值、状态变化与错误；
3. 对产生错误的程序给出第 14 节规定的错误类型；

4. 保持本文规定的求值顺序与作用域规则。

实现可以对资源上限、诊断文本、调试信息、执行策略作出选择。这些选择应当保持程序的语义结果。

1.4 版本

本规范版本号为 0.1。版本号由主版本、次版本组成。主版本变化表示语义或文法的兼容性中断；次版本变化表示兼容扩充与澄清。

1.5 统一表达式模型

JC2 的程序、块、声明、控制结构与模式绑定共享同一表达式语义：

1. 每个表达式产生一个值，或执行一次控制转移；
2. 程序与块是表达式序列，序列位置确定求值顺序；
3. 声明形式产生对应的绑定值或类型值；
4. 控制形式产生分支值、`none` 或控制转移；
5. 模式绑定是表达式，并产生被匹配的值。

表达式在序列中的位置确定求值顺序；表达式本身始终按统一规则求值。

2 源文本

2.1 编码

JC2 源文件采用 UTF-8 编码。源文件可以包含字节顺序标记；字节顺序标记在词法分析前移除。源文本按 Unicode code point 处理。

UTF-8 解码按最长合法序列进行。解码失败时，在出现位置产生 `SyntaxError`。源文件中的 NUL 字符按普通字符保留，诊断文本可以转义显示。

2.2 行与空白

换行符为 `U+000A`。回车符 `U+000D` 在词法分析前规范化为换行符。空格、制表符、换页符与换行符构成空白。空白分隔相邻词法单元。

词法分析在圆括号与方括号内部忽略换行；花括号内部保留换行。换行作为表达式序列的分隔符；连续换行与分号等价。

2.3 注释

行注释以 `//` 开始，延续到行尾。块注释以 `/*` 开始，以 `*/` 结束；块注释可以跨行，并支持嵌套。注释在词法分析阶段结束，注释内容进入诊断信息的方式由实现决定。

2.4 词法单元

词法单元分为以下类别：

- 标识符与关键字；
- 数字字面量、字符串字面量与虚数字面量；
- 运算符与分隔符；
- 换行、文件结束。

词法分析采用最长匹配。标识符与关键字区分大小写。

扫描从文件起始位置开始，按当前字符选择产生式：

1. 空白与注释直接跳过；
2. 字母或下划线开始标识符扫描；
3. 数字开始数字字面量扫描；
4. 引号或前缀 `r`、`f` 开始字符串扫描；
5. 反引号开始反引号标识符扫描；
6. 运算符按最长匹配识别；
7. 上述规则均失败时，该字符产生 `SyntaxError`。

2.5 标识符与关键字

标识符由字母、数字与下划线组成，首字符为字母或下划线。

反引号标识符以反引号包围内容，内容作为标识符名：

```
`class` = 1
import `1`
```

反引号标识符的内容可以包含关键字、数字与标点；内容为空、包含换行或以 `$` 开头时产生 `SyntaxError`。反引号标识符产生 `IDENTIFIER` 词法单元，其 `lexeme` 为反引号内部的文本。

以下保留字具有关键字含义：

```
if else while for in is as break continue return
local ref state const static delete throw try catch import include
match macro syntax quote defer
class trait with extends namespace enum super self
true false none
```

关键字可以出现在成员名位置时按成员名处理；该位置由文法规定。标识符的 Unicode 类别包括字母类与连接符下划线；组合字符可以出现在首字符之后。

2.6 数字字面量

数字字面量包括：

- 十进制整数，例如 `42`、`-7`；
- 十六进制整数，前缀 `0x` 或 `0X`；
- 二进制整数，前缀 `0b` 或 `0B`；
- 八进制整数，前缀 `0o` 或 `0O`；
- 十进制浮点数，例如 `3.14`、`1.0e-9`；
- 虚数字面量，后缀 `i`，例如 `2i`。

数字字面量的值由第 5.4 节规定。整数字面量超出机器字长时形成任意精度整数。数字字面量从前导数字开始；符号由一元运算符处理。浮点字面量允许指数部分，例如 `6.02e23`、`1E-9`；十六进制、二进制与八进制字面量表示整数。

数字可以用下划线分组，例如 `10_000_000`、`0xFF_FF`。下划线的两侧各有一个数字；连续下划线、前缀后的下划线、小数点后的下划线以及末尾下划线均产生 `SyntaxError`。

2.7 字符串字面量

字符串字面量包括：

- 双引号字符串：`"..."`；
- 单引号字符串：`'...'`；
- 三引号字符串：`"""..."""` 与 `'''...'''`；
- 原始字符串：`r"..."` 与 `r'...'`；
- 自定义分隔符原始字符串：`r"TAG(...)TAG"` 与 `r'TAG(...)TAG'`；
- 格式化字符串：`f"..."` 与 `f'...'`。

双引号字符串支持单引号字符；单引号字符串支持双引号字符。三引号字符串保留换行。原始字符串按字面保留反斜杠。自定义分隔符原始字符串以 `r"TAG(` 开始，以 `)TAG"` 结束；中间的 `TAG` 为标识符序列。`TAG` 为空时仍按括号定界：`r"(abc)" == "abc"`。普通原始字符串写作 `r"abc"`，不带括号。格式化字符串的插值规则由第 13.1 节规定。

字符串缺少闭合引号时，在起始位置产生 `SyntaxError`。普通字符串中的转义序列按上表解码；其他转义保留反斜杠与字符本身。格式化字符串的插值表达式在当前作用域求值；插值结果的字符串形式按第 13.1 节规定。

转义序列包括：

序列	含义	序列	含义
`\n`	换行	`\t`	制表
`\\`	反斜杠	`\"`	双引号
`\'`	单引号	`\r`	回车
`\0`	空字符	`\a`	响铃
`\b`	退格	`\f`	换页
`\v`	垂直制表	`\xNN`	十六进制字节
`\NNN`	八进制字节		

2.8 运算符与分隔符

运算符与分隔符包括：

```
+ - * / ~/ \ ^ % = == != < <= > >= <=>
+= -= *= /= %= ^= &= |= ^^= <<= >>= ~/= \=
& | ^^ ~ ! && ||
<< >> <: |>
( ) [ ] { } , ; : . ? @ $
... -> =>
```

`^` 表示幂运算；`^^` 表示按位异或或集合对称差。`<:` 表示子集或子类关系。

2.9 词法文法

```
source-file   = { whitespace | comment | token } EOF ;
whitespace    = " " | "\t" | "\f" | newline ;
newline       = "\n" | "\r\n" | "\r" ;
comment       = line-comment | block-comment ;
line-comment  = "//" { any-char-except-newline } newline ;
block-comment = "/*" { block-comment | comment-text } "*/" ;
comment-text  = ? any char except "*/" ? ;
```

```
identifier    = plain-identifier | quoted-identifier ;
plain-identifier = letter { letter | digit | "_" } ;
quoted-identifier = "`" { any-char-except-backtick-and-newline } "`" ;
letter        = "A".."Z" | "a".."z" | "_" | unicode-letter ;
number        = decimal-integer | hex-integer | bin-integer
              | oct-integer | float-number | imaginary-number ;
decimal-integer = digit { [ "_" ] digit } ;
hex-integer   = ( "0x" | "0X" ) hexdigit { [ "_" ] hexdigit } ;
bin-integer   = ( "0b" | "0B" ) bindigit { [ "_" ] bindigit } ;
oct-integer   = ( "0o" | "0O" ) octdigit { [ "_" ] octdigit } ;
float-number  = digit { [ "_" ] digit } "." digit { [ "_" ] digit } [
    exponent ]
              | digit { [ "_" ] digit } exponent ;
exponent      = ( "e" | "E" ) [ "+" | "-" ] digit { digit } ;
imaginary-number = ( decimal-integer | hex-integer | bin-integer
              | oct-integer | float-number ) "i" ;
string        = quoted-string | triple-string | raw-string | f-string ;
raw-string    = ordinary-raw | raw-triple | delimited-raw ;
ordinary-raw  = 'r"' { any-char-except-quote } '"' | "r'" { any-char-except
    -quote } "'" ;
raw-triple    = 'r"""' { any-char } '"""' | "r'''" { any-char } "'''" ;
delimited-raw = 'r"' [ raw-delimiter ] "(" { any-char } ")" [ raw-delimiter
     ] '"'
              | "r'" [ raw-delimiter ] "(" { any-char } ")" [ raw-delimiter
     ] "'" ;
raw-delimiter = identifier ;
f-string      = "f" ( '"' { any-char } '"' | "'" { any-char } "'"
              | '"""' { any-char } '"""' | "'''" { any-char } "'''" ) ;
```

2.10 编译器指令

行首的 # 引导编译器指令。指令名与参数延续到行尾，参数按空白裁剪。指令在源文件解析前按出现顺序处理：

指令	语义
`#!`	脚本首行标记，按空操作处理
`#version op V`	要求实现版本满足 `op V`；默认 `op` 为 `>=`
`#platform P`	要求运行平台为 `windows`、`linux` 或 `macos`
`#warning M`	输出警告 `M`
`#error M`	产生 `ParserError`，消息为 `M`

`#version` 的比较运算符包括 `>=`、`<=`、`==`、`=`、`>`、`<`。其他指令名产生 `ParserError`。指令只在行首出现；行首空白计入行首位置。

2.11 词法错误与位置

词法单元携带起始字节位置、起始行号与源文件标识。诊断位置按起始行号与字符偏移计算。词法错误在出现位置产生，并携带源文件、行号与消息。

条件	错误
UTF-8 解码失败	`SyntaxError`
字符属于词法单元之外	`SyntaxError`
字符串或注释缺少闭合符	`SyntaxError`
反引号标识符缺少闭合符	`SyntaxError`
数字字面量格式错误	`SyntaxError`
编译器指令名属于其他值	`ParserError`

词法分析在错误点记录诊断，并继续扫描到行尾或文件结束，以便同一源文件报告多个独立错误。

3 文法

3.1 记法

本文使用扩展巴科斯范式（EBNF）：

形式	含义
`A B`	序列
`A | B`	选择
`[ A ]`	可选
`{ A }`	零次或多次
`( A )`	分组
`"x"`	终结符

文法以抽象语法为目标；空白、注释与换行在词法阶段结束。运算符优先级在第 6.4 节以表的形式规定。

3.2 程序与表达式序列

程序与块都是表达式序列：

```
program        = { expression } EOF ;
block          = "{" { expression } "}" ;
```

表达式序列按书写顺序求值；序列的值为最后一个表达式的值，空序列的值为 `none`。

3.3 声明表达式

声明形式是表达式，并产生对应的值：

```
binding        = [ "const" ] [ "local" | "ref" | "state" ]
                 binding-target [ ":" expression ]
                 [ "=" expression ] ;
binding-target = identifier | pattern ;
function-binding = [ "const" ] [ "local" | "ref" | "state" ]
                 identifier "(" [ params ] ")"
                 [ "->" expression ] "=" function-body ;
function-body  = expression | block ;

name-spec      = [ "$" ] identifier | string ;
import-expr    = "import" [ "@" ] ( module-path | string ) ;
module-path    = identifier { "." identifier } ;
include-expr   = "include" expression ;
delete-expr    = "delete" [ "@" ] identifier { "," identifier } ;
namespace-expr = "namespace" [ name-spec ] block ;
class-expr     = "class" [ name-spec ] [ "extends" expression ]
                 [ "with" trait-list ] class-body ;
trait-expr     = "trait" [ name-spec ] [ "with" trait-list ] class-body ;
trait-list     = expression { "," expression } ;
class-body     = "{" { class-member } "}" ;
class-member   = { member-modifier } ( field-decl | method-decl ) ;
member-modifier = "static" | "local" | "const" ;
field-decl     = member-name [ "=" expression ] ;
method-decl    = member-name "(" [ params ] ")" [ "->" expression ]
                 [ "=" function-body ] ;
member-name    = identifier | keyword ;
```

```
enum-expr      = "enum" [ name-spec ] "{" { enum-member } "}" ;
enum-member    = ( identifier | keyword ) [ "=" expression ] ;
macro-def      = "macro" [ "$" ] identifier "(" [ macro-params ] ")" "="
   function-body ;
macro-params   = macro-param { "," macro-param } ;
macro-param    = [ "..." ] identifier ;
syntax-def     = "syntax" [ "$" ] identifier "(" identifier ")" "="
   function-body ;
context-self-expr = "class" | "namespace" ;
```

name-spec 为标识符时建立同名绑定；为字符串时设置定义名，绑定由外围赋值完成。省略 name-spec 时形成匿名定义，左花括号与定义关键字位于同一行。

macro-def 与 syntax-def 的函数体采用 function-body，可以是一个表达式或一个块。

3.4 控制表达式

控制结构是表达式，并产生分支值或控制转移：

```
if-expr        = "if" "(" expression ")" expression
                 [ "else" expression ] ;
while-expr     = "while" "(" expression ")" expression ;
for-expr       = "for" "(" [ expression ] ";" [ expression ] ";"
                 [ expression ] ")" expression ;
for-in-expr    = "for" "(" for-pattern "in" expression ")" expression ;
for-pattern    = [ "const" ] [ "local" ] pattern ;

match-expr     = "match" "(" expression ")" "{" match-arms "}" ;
match-arms     = match-arm { "," match-arm } ;
match-arm      = branch-patterns [ "if" "(" expression ")" ] "=>"
   expression ;

try-expr       = "try" expression [ catch-clause ] ;
catch-clause   = "catch" ( "(" branch-patterns ")" expression
                         | "{" match-arms "}" ) ;
defer-expr     = "defer" expression ;
return-expr    = "return" [ expression ] ;
break-expr     = "break" ;
continue-expr  = "continue" ;
throw-expr     = "throw" expression ;
```

3.5 表达式

```
expression     = sequence ;
sequence       = assignment { "," assignment } ;
assignment     = ternary [ assign-op assignment ] ;
assign-op      = "=" | "+=" | "-=" | "*=" | "/=" | "%="
               | "~/=" | "\\="
               | "^=" | "&=" | "|=" | "^^=" | "<<=" | ">>=" ;

ternary        = pipe [ "?" expression ":" ternary ] ;
pipe           = logical-or { "|>" pipe-step } ;
pipe-step      = "." ( identifier | keyword ) [ "(" [ arguments ] ")" ]
               | logical-or ;
logical-or     = logical-and { "||" logical-and } ;
logical-and    = comparison { "&&" comparison } ;
comparison     = bitwise-or { compare-op bitwise-or } ;
compare-op     = "==" | "!=" | "<" | "<=" | ">" | ">="
               | "<=>" | "in" | "is" | "<:" ;

bitwise-or     = bitwise-xor { "|" bitwise-xor } ;
bitwise-xor    = bitwise-and { "^^" bitwise-and } ;
bitwise-and    = shift { "&" shift } ;
shift          = additive { ( "<<" | ">>" ) additive } ;
additive       = multiplicative { ( "+" | "-" ) multiplicative } ;
multiplicative = unary { ( "*" | "/" | "~/" | "\\" | "%" ) unary } ;
unary          = ( "!" | "-" | "+" | "~" ) unary | power ;
power          = as-expr [ "^" unary ] ;
as-expr        = postfix { "as" postfix } ;
postfix        = primary { call-suffix | index-suffix | member-suffix } ;

call-suffix    = "(" [ arguments ] ")" ;
index-suffix   = "[" index-arg { "," index-arg } "]" ;
index-arg      = expression
               | [ expression ] ":" [ expression ] [ ":" [ expression ] ] ;
member-suffix  = "." ( identifier | keyword ) ;

primary        = literal | identifier | "(" expression ")"
               | lambda | list-literal | set-literal
               | dict-literal | matrix-literal
               | list-comprehension | matrix-comprehension
```

```
              | set-comprehension | dict-comprehension
              | quote-expr
              | macro-call | block
              | binding | function-binding | import-expr
              | include-expr
              | delete-expr
              | context-self-expr
              | class-expr | trait-expr | namespace-expr
              | enum-expr | macro-def | syntax-def
              | if-expr | while-expr | for-expr | for-in-expr
              | match-expr | try-expr | defer-expr
              | return-expr | break-expr | continue-expr
              | throw-expr ;
```

3.6 字面量与构造

```
list-literal   = "@[" [ matrix-row { ";" matrix-row } ] "]" ;
set-literal    = "@{" [ expression { "," expression } ] "}" ;
dict-literal   = "{" [ dict-entry { "," dict-entry } ] "}" ;
dict-entry     = expression ":" expression
               | "..." expression
               | identifier ;
matrix-literal = "[" [ matrix-row { ";" matrix-row } ] "]" ;
matrix-row     = expression { "," expression } ;
lambda         = "(" [ params ] ")" "=>" ( expression | block ) ;
quote-expr     = "quote" ( block | expression ) ;
macro-call     = "@" identifier [ "(" [ arguments ] ")" ] [ block ] ;
comp-clause    = "for" "(" for-pattern "in" expression ")"
                 { "if" "(" expression ")" } ;
comp-clauses   = comp-clause { comp-clause } ;
list-comprehension   = "@[" expression comp-clauses "]" ;
matrix-comprehension = "[" expression comp-clauses "]" ;
set-comprehension    = "@{" expression comp-clauses "}" ;
dict-comprehension   = "{" expression ":" expression comp-clauses "}" ;
```

3.7 参数与实参

```
params         = param { "," param } [ ";" param { "," param } ]
```

```
                | ";" param { "," param } ;
param           = [ "const" ] [ "ref" | "local" | "state" ]
                  ( "..." identifier | pattern [ ":" expression ]
                    [ "=" expression ] ) ;
arguments       = argument { "," argument } [ ";" argument { "," argument }
   ]
                | ";" argument { "," argument } ;
argument        = expression | identifier "=" expression
                | "..." expression ;
```

参数列表中的 ; 分隔位置参数与仅关键字参数。...name 在 ; 前收集剩余位置参数，在 ; 后收集剩余关键字参数。

3.8 模式

```
pattern         = primary-pattern [ "=" expression ] ;
primary-pattern = "_" | variable-pattern | literal
                | list-pattern | matrix-pattern
                | dict-pattern | rest-pattern | dynamic-assert-pattern
                | [ "local" | "ref" | "state" | "const" ] primary-pattern ;

variable-pattern = identifier [ ":" expression ] ;
rest-pattern    = "..." identifier [ ":" expression ] ;
list-pattern    = "@[" [ pattern { "," pattern } ] "]" ;
matrix-pattern = "[" [ pattern-row { ";" pattern-row } [ ";" ] ] "]" ;
pattern-row     = pattern { "," pattern } ;

dict-pattern    = "{" [ field-pattern { "," field-pattern }
                  [ "," rest-pattern ] ] "}" ;
field-pattern   = [ "const" ] [ "local" | "ref" | "state" ] identifier
                  [ ":" pattern | "=" expression ]
                | string ":" pattern ;
dynamic-assert-pattern = "(" expression ")" ;
branch-patterns = pattern { "," pattern } ;
```

矩阵模式使用分号分隔行，例如 [X; T]。类型模式写作 identifier ":" expression，例如 e: TypeError。默认模式写作 pattern "=" expression。或模式在分支层以逗号连接，例如 1, 2 => ... 与 e: TypeError, e: ValueError => ...。竖线用于按位或与集合并；分支模式的组合符为逗号。动态断言模式写作 (expression)，按表达式真值匹配。

模式的具体匹配规则由第 9 节规定。

3.9 句法歧义消解

以下歧义按固定规则消解：

1. `[` 位于表达式位置时开始矩阵字面量；位于后缀位置时开始索引或切片。
2. `@[` 开始 list 字面量；`@{` 开始 set 字面量；`@name` 开始宏调用。
3. `{` 位于表达式位置时按块解析；键值对形式按 dict 字面量解析。
4. 匿名 class、trait、namespace、enum 的左花括号与关键字位于同一行。
5. 形如 `a = b` 的实参按关键字实参解析；括号内的 `(a = b)` 按赋值表达式解析。
6. 调用实参列表中的逗号分隔实参；括号外的逗号进入逗号序列表达式。
7. `|>` 右侧以 `.name`、`.method(args)` 或可调用表达式为管道步骤。
8. `<=>` 在连续比较链中出现时产生 `SyntaxError`。

3.9.1 dict 字面量与块的区分

花括号的解析分两步：先扫描配对花括号内部，记录深度、冒号、分号、赋值与三元运算符；再按以下规则分类：

1. `{}` 是空 dict。
2. 深度零处的冒号产生 dict 字面量，例如 `{a: 1}`、`{"k": 2}`。
3. 深度零处的分号产生块，例如 `{a; b}`。
4. 深度零处的赋值产生块，例如 `{a = 1}`。
5. 深度零处的三元运算符产生块，例如 `{a ? b : c}`。
6. 仅由标识符与逗号组成的序列产生简写 dict，例如 `{a, b}`，等价于 `{"a": a, "b": b}`。
7. 简写条目与显式键值条目可以混合，例如 `{a, b, c: 3}`。
8. `...expr` 条目展开 dict 并合并其键值，例如 `{a, ...d}`。
9. 其余情况按块解析。

分类只检查配对花括号的最外层；括号、方括号与嵌套花括号内部的冒号与分号参与嵌套结构。块内的相邻表达式之间由换行或分号分隔；同一行相邻标识符产生 `SyntaxError`。dict 键的规则：裸标识符键使用标识符文本；字符串键使用字面量；其他表达式键按表达式求值。简写键使用标识符文本，值读取同名变量。

```
a = 1
b = 2
{a: 3}          // {"a": 3}
{a, b}          // {"a": 1, "b": 2}
{}              // {}
{a = 4}         // 块，值为 4
{a; b}          // 块，值为 2
{a ? b : 3}     // 块，值为 2
```

3.10 文法的静态约束

文法接受的结构还需要满足以下静态约束：

1. 赋值左侧为变量、成员、索引、切片或解构模式，并且可写。
2. `ref` 目标为已有 lvalue；`const ref` 目标为已有 lvalue。
3. `local`、`ref`、`state` 三者互斥；`const` 可以与任一者组合。
4. 参数列表中 `;` 出现至多一次；`...rest` 与 `...kw` 位于各自区域末尾。
5. 默认参数位于必填位置参数之后；仅关键字参数位于 `;` 之后。
6. pattern 中 `...rest` 出现至多一次；rest 目标可以位于 list 或 matrix 行的任意位置，捕获固定头尾之间的元素。
7. class 体成员名在同类中唯一；重复成员产生 `SyntaxError`。
8. enum 成员名在 enum 内唯一；重复成员产生 `SyntaxError`。
9. 匿名定义的关键字与左花括号位于同一行。

4 名字、绑定与作用域

4.1 绑定

以下形式产生绑定：

- `import name` 产生名字 `name`；
- `import "path"` 产生值，用于表达式；
- `class Name`、`trait Name`、`namespace Name`、`enum Name` 产生名字；
- 函数声明产生名字；

- 变量声明与赋值产生名字或更新既有绑定；
- 参数、模式目标、循环变量与 catch 变量产生局部绑定。

每个绑定包含名字、种类、作用域、初始化状态与只读属性。

绑定形式	种类	作用域	初始化时点
变量绑定	值绑定	声明所在块或提升后的函数/namespace	声明求值时
函数绑定	函数值	声明所在块	定义求值时
class / trait / namespace / enum	class 对象或 namespace 值	声明所在块	定义求值时
import	模块值	声明所在块	导入求值时
参数	值或引用参数	函数体	调用绑定时
模式目标	值或引用绑定	模式所在块	匹配提交时
循环变量	值绑定	循环体或提升后的作用域	每轮迭代开始时
catch 变量	值绑定	catch 体	错误匹配时

class 数据属性省略初始化器时进入待初始化状态。读取待初始化属性产生 `RuntimeError`。普通 `local`、`ref`、`state` 声明省略初始化器时捕获外层绑定，不进入待初始化状态。只读绑定在首次提交后保持只读。

4.1.1 重声明与遮蔽

同一作用域内的显式重声明产生 `SyntaxError`：

```
local x = 1
local x = 2       // SyntaxError: Variable 'x' has already been declared
```

普通赋值更新既有绑定。内层作用域可以声明同名绑定，内层绑定遮蔽外层绑定。声明自身采用先求值右侧、后注册左侧的顺序，因此 `local x = x` 捕获外层 `x`。独立声明 `local x` 捕获外层 `x` 的当前值；外层 `x` 缺失时产生 `RuntimeError`。

局部名字在声明点之后可用；在声明点之前读取产生 `SyntaxError`。函数名与其他全局或命名空间绑定在调用时解析，因此后定义的函数可以从先定义的函数体调用。

4.1.2 局部捕获与闭包

闭包捕获局部变量时，变量存储随闭包延长，名字本身在块结束时退出作用域。捕获后的写入作用于同一存储；闭包之间共享该存储。

4.2 作用域

作用域由以下结构建立：

- 源文件与 namespace 体；
- 函数体与 lambda 体；
- 块、循环体、catch 体；
- class 与 trait 体；
- comprehension 与 match 分支。

名字解析选择最近的外层绑定。局部绑定在声明点之后可见。函数体中的自由名字按定义位置的词法环境解析。

4.2.1 解析顺序

自由名字按以下顺序解析：

1. 当前块的局部绑定；
2. 当前函数的参数与局部绑定；
3. 外围函数的捕获绑定；
4. 外围 namespace 的 public 绑定；
5. 顶层全局绑定；
6. 全局内建名。

成员访问 `a.name` 在 `a` 的字段中解析；字段缺失时沿 `a` 的原型或 class 链解析。namespace 缺少成员时按 namespace 规则处理；package 成员缺失时产生错误。`self.name` 与 `Target.name` 使用目标对象的字段表。

4.3 提升

隐式赋值在以下位置提升到最近的函数或 namespace 作用域：

- 函数体顶层；
- namespace 体顶层；
- 顶层程序。

显式 `local` 绑定保持在当前块。循环变量默认提升；`for (local x in ...)` 保持局部。match 分支变量与 comprehension 变量保持局部。

顶层程序的普通赋值进入全局绑定。函数体内的普通赋值进入函数级 auto-local 绑定。块、if、for、while 的普通赋值继续路由到所属函数或全局；`local` 将绑定限制在当前块。自赋值声明 `x = x` 在注册新绑定前求值右侧，右侧读取外层 `x`。

4.4 修饰符

变量修饰符组合如下：

形式	绑定位置	语义
`local`	当前块	块级作用域
`ref`	已有 lvalue	引用绑定
`state`	函数或闭包	每次调用的持久状态
`const`	当前绑定	只读绑定
`const ref`	已有 lvalue	只读引用

`local`、`ref`、`state` 三者互斥。`const` 可以与任一者组合。`state` 出现在顶层、namespace 体或 class 体时产生错误。`const ref` 带初始化器时产生错误。

4.4.1 ref 绑定

`ref x` 绑定到外层已有 `x`；外层 `x` 缺失时产生 `RuntimeError`。通过 `ref` 绑定的读写直接作用于外层变量。`ref x = expr` 先求值 `expr`，再写入外层 `x`。`ref x += expr` 在外层变量上执行读改写。函数参数 `ref x` 要求调用实参为 lvalue；字面量与临时值产生 `TypeError`。

4.4.2 state 绑定

`state x` 在定义时捕获外层 `x`；外层 `x` 缺失时在定义时产生 `RuntimeError`。`state x = expr` 在第一次调用时求值 `expr` 一次，之后复用该值。`state x += expr` 在定义时捕获外层 `x`，每次调用累加 `expr`。`state x: T` 在捕获后执行类型断言。`state` 绑定属于闭包实例；各闭包实例各有独立状态。

4.4.3 const 与删除

`const` 绑定在初始化后保持只读；赋值与 `delete` 均产生 `RuntimeError`。系统常量可以通过导入的 `sys` 命名空间恢复：`import sys; sys.resetConst()`；用户 `const` 绑定保持只读。

`delete a, b` 删除变量绑定；`delete @macro` 删除编译期宏定义。删除表达式返回 `none`；`const` 绑定删除产生 `RuntimeError`。删除操作接受变量名列表，索引目标通过容器方法删除。内建函数可以被用户绑定遮蔽；`delete` 删除遮蔽绑定，内建函数本身继续存在。

4.5 模块与 include

`import pkg.sub` 将根名 `pkg` 绑定到模块对象。模块对象包含根字段与子模块字段。重复导入返回同一对象。导入失败时，该次导入产生错误；后续导入重新执行。

`include expr` 将 namespace 的 public 字段复制到当前命名空间作用域。目标为执行栈中最内层正在构造的 namespace；栈中没有该目标时产生 `RuntimeError`。复制过程保持原子性：任一冲突产生错误，目标 namespace 保持导入前状态。

5 类型与值

5.1 值域

JC2 的值包括：

值域	说明
`int`	任意精度整数
`float`	双精度浮点数
`fraction`	精确有理数
`complex`	复数
`symbolic`	符号表达式
`string`	UTF-8 字符串
`bool`	真值
`none`	空值
`list`、`dict`、`set`	容器
`matrix`	实、复或符号矩阵
`function`	闭包与原生函数
`type`	类型对象
`slice`	切片描述符
`namespace`	模块命名空间
`class`	class 对象；trait 定义是 class 对象的特例
`instance`	class 实例

`slice` 是一等值，包含 `start`、`end`、`step` 三个可读字段。`spread`、`super`、`upvalue` 属于求值与闭包内部形式，用户通过对应语法使用。`int` 覆盖机器整数与任意精度整数。

5.2 类型

类型由类型对象表示。程序可以用 `type(x)` 取得值的类型。primitive 类型的测试形式为 `type(x) == T` 与 `x in T`。class 实例与组合 trait 的实例使用 `isinstance(x, C)`

与 `x in C` 做类型测试。`isinstance(x)` 检查任意 class 实例。

联合类型 `A | B` 表示值属于 `A` 或 `B`。类型注解在声明处执行一次检查；函数返回注解在每次调用返回时检查。

5.2.1 类型对象与驻留

类型是原生对象。类型对象全局驻留：结构相同的类型指向同一对象。因此 `type(5) == int` 与 `type(5) is int` 都按 identity 比较。`type(int)` 是 `type`；`type(Point)` 是 `class_type`；`type(Point())` 是 `Point`。

类型可以用位运算组合：

```
NumType = int | float
(int | string) & int      // int
int & string              // never
int | Point               // int | Point
```

`|` 构造并集，`&` 构造交集；类型集合自动去重并化简。`any` 是类型全集，`never` 是空集。`never <: T` 对每个 `T` 成立。

5.2.2 原子类型与联合类型

原子类型包括：

```
int float string bool none_type list dict set fraction complex
symbolic realmatrix complexmatrix symmatrix function class_type
instance namespace_type type slice
```

联合类型由原子类型构造，包括：

```
exact  = int | fraction | symbolic | bool
matrix = realmatrix | complexmatrix | symmatrix
real   = int | float | fraction | bool
number = int | float | fraction | complex | bool
```

5.2.3 类型注解与 as 断言

类型注解写在绑定、参数、返回与解构目标上：

```
x: int = 10
f(x: int | string) -> int = ...
[a: int, b: string] = data
d.a: string = "x"
l[0]: int = 5
```

注解在绑定点执行一次检查；类型条件失败时产生 TypeError。as 是值断言表达式：求值左侧，检查类型，返回原值。

```
x as int
(a + b) as int
x as (int | string)
```

as 比所有二元运算符绑定更紧，仅比后缀访问松；连续 as 左结合。as 表达式本身不需要括号。断言的类型为联合类型时，联合类型表达式需要括号：x as (int | string)。as 检查类型并保留值；类型转换由类型对象调用完成。

独立声明 local x: int 先捕获外层 x，再执行 int 断言。state x: int 在定义时捕获并断言；ref x: int 断言外层当前值。

5.2.4 类型对象调用

类型对象可以调用，执行构造或转换：

```
int(3.14)          // 3
float(5)           // 5.0
complex(1, 2)      // 1+2i
string(42)         // "42"
list(1, 2, 3)      // @[1, 2, 3]
dict("a", 1)       // {"a": 1}
set(1, 2, 2, 3)    // @{1, 2, 3}
slice(1, 5, 2)     // slice(1, 5, 2)
matrix(2, 2, 1, 2, 3, 4)
```

类型对象是全局变量；覆盖后可用导入的 sys 命名空间恢复：import sys; sys.resetType()。

5.2.5 行为谓词

行为谓词检查能力；精确类型检查使用 type：

```
isiterable(x)    // 可迭代
iscallable(x)    // 可调用
isindexable(x)   // 可索引
ishashable(x)    // 可哈希
istrait(x)       // trait 定义
```

isapprox(a, b, rtol, atol) 使用 $|a-b| \le \max(\text{atol}, \text{rtol} \cdot \max(|a|,|b|))$；默认 rtol = 1e-9、atol = 0。矩阵按同形状逐元素比较。

5.3 真值

以下值为假：

类别	假值
数值	`0`、`0.0`、零 fraction、零 symbolic
字符串	`""`
空值	`none`
容器	空 list、空 dict、空 set
矩阵	零元素矩阵

其余值为真。条件、`&&`、`||` 与 `!` 按该规则求值。`&&` 与 `||` 返回严格布尔值。

5.4 数值

数值类型按以下规则提升：

运算	结果类型
`int` 与 `int`	`int`
`int` 与 `float`	`float`
`int` 与 `fraction`	`fraction`
`int` 与 `complex`	`complex`
`int` 与 `symbolic`	`symbolic`

整数 `/` 产生精确 `fraction`。`~/` 向零截断。除法、模运算与 fraction 构造在零分母处产生 `MathError`。浮点溢出产生 `inf`。任意精度整数转换为浮点数超出范围时产生 `MathError`。

`nan()` 产生规范 NaN 值。NaN 与任何值比较相等的结果为假；NaN 可以作为 dict key 或 set 元素；NaN 参与 `<=>` 时产生 `TypeError`。`inf()` 与 `-inf()` 参与普通浮点比较与算术。

5.5 相等、同一性与排序

相等按值定义，规则按类型分组：

- 数值：bool、int、BigInt、float、fraction、complex 与 symbolic 按数值比较；`1 == 1.0 == true`，`-0.0 == 0.0`，NaN 与任何值（包括自身）相等的结果为假；
- 字符串按 code point 序列比较；
- slice 按 `start`、`end`、`step` 比较；

- list 按长度与逐元素相等比较；
- dict 按键集合与对应值相等比较，键顺序不影响结果；
- set 按元素集合相等比较，顺序与重复次数不影响结果；
- 矩阵按形状与逐元素数值相等比较；实数、复数、符号三种矩阵表示可以互比；
- class、trait、namespace、enum、function、type 按 identity 比较；
- 实例提供 `__eq__` 时使用该方法；普通实例按 identity 比较；两个冻结实例在同类且公开属性与私有属性值相等时相等。

`is` 比较对象 identity；相等不要求 `is`。`<=>` 返回 `-1`、`0`、`1`，其定义域为数值、字符串与实现 `__cmp__` 的实例。

5.5.1 哈希规则

`hash(x)` 返回整数。相等值具有相同 hash；hash 在对象生命周期内保持稳定。

可哈希值包括：

- none、bool、int/BigInt、float、fraction、complex、symbolic；
- string、slice、matrix；
- type、class、function、namespace、enum；
- 冻结 list、冻结 dict、冻结 set，且其元素或值全部可哈希；
- 提供 `__hash__` 的实例，以及属性全部可哈希的冻结实例。

可变 list、dict、set，以及没有 `__hash__` 的非冻结实例，作为 dict key 或 set 元素时产生 `TypeError`。数值 hash 对跨表示相等值归一化：`1`、`1.0`、`true` 具有相同 hash；`-0.0` 与 `0.0` 具有相同 hash。NaN 可以参与 hash，NaN 与自身相等的结果保持为假。`__hash__` 的返回值参与 hash 计算；相等实例的 `__hash__` 返回相等结果。

```
1 == true             // true
1 == 1.0              // true
nan() == nan()        // false
hash(1) == hash(1.0)  // true
hash(1) == hash(true) // true
```

5.6 复制与冻结

`copy(x)` 产生深拷贝,并保持源锁状态。`copy(x, false)` 产生深拷贝,结果完全可写。`copy(x, true)` 产生深拷贝，结果完全只读。只读容器的修改操作产生 `RuntimeError`。

6 表达式求值

6.1 求值关系

表达式求值使用四元配置：

$$\langle e,\ E,\ H,\ D\rangle \Downarrow \langle v,\ E',\ H',\ D'\rangle$$

其中 `e` 为待求值表达式，`E` 为词法环境，`H` 为异常处理器栈，`D` 为 defer 栈。求值结果包括：

- 值 `v`，同时给出更新后的环境、处理器栈与 defer 栈；
- 控制转移：`return v`、`break`、`continue`；
- 错误 `throw e`。

求值关系按表达式结构归纳。子表达式求值产生的控制转移与错误传播到父表达式；defer 在当前块退出时执行。

6.2 基本规则

1. 字面量求值为其常量值，环境维持原值。
2. 变量读取在当前环境查找绑定；绑定处于待初始化状态时产生 `RuntimeError`。
3. 赋值先求值右侧，再按赋值规则更新目标；表达式值为写入值。
4. 表达式序列从左到右求值，值为最后一个表达式的值。
5. 二元运算先求值左操作数，再求值右操作数，然后应用运算符。
6. 条件表达式求值条件，再求值选中分支。
7. 调用先求值被调用值，再按第 8 节绑定参数并执行函数体。

子表达式产生错误时，父表达式的剩余部分停止求值，错误进入当前处理器栈。子表达式产生控制转移时，剩余部分停止求值，转移沿块边界传播。

6.3 求值顺序

表达式按以下顺序求值：

1. 调用目标表达式；
2. 位置实参从左到右；
3. 关键字实参从左到右；

4. spread 实参在出现位置展开。

二元运算先求值左操作数，再求值右操作数。`&&` 与 `||` 短路：左侧结果决定右侧求值条件。三元表达式只求值选中分支。chained comparison 的中间表达式只求值一次。

6.4 优先级与结合性

级别	运算符	结合性
1	后缀 `()`、`[]`、`.`	左
2	`as`	左
3	前缀 `!`、`-`、`+`、`~`	右
4	`^`	右
5	`*`、`/`、`~/`、`\`、`%`	左
6	`+`、`-`	左
7	`<<`、`>>`	左
8	`&`、`^^`、`|`	左
9	`==`、`!=`、`<`、`<=`、`>`、`>=`、`in`、`is`、`<:`	链式
9	`<=>`	独立
10	`&&`	左
11	`||`	左
12	`|>`	左
13	`? :`	右
14	赋值（含 `~/=`、`\=`）	右
15	逗号序列	左

`|>` 将左侧值作为右侧表达式的接收者。右侧形式包括：

- `.field`：读取左侧值的属性，等价于 `left.field`；
- `.method(args)`：调用左侧值的方法；
- 函数名、带占位符调用或 lambda：以左侧值为参数调用。

```
inc(x) = x + 1
5 |> inc              // 6
add(a, b) = a + b
2 |> add(3, _)        // 5
obj |> .field         // obj.field
obj |> .method()      // obj.method()
```

逗号序列的优先级低于赋值，例如 `a = 1, b = 2` 形成两个赋值表达式。

位运算的优先级高于比较运算，因此 `1 | 2 == 3` 等价于 `(1 | 2) == 3`。`-2 ^ 2` 等价于 `-(2 ^ 2)`。`2 ^ 3 ^ 2` 等价于 `2 ^ (3 ^ 2)`。

6.5 算术

整数运算保持任意精度。`~/` 与 `%` 使用截断除法：

```
(-7) ~/ 2 = -3, (-7) % 3 = -1, 7 % (-3) = 1
```

`\` 为左除：`a \ b` 等价于 `b / a`。矩阵的左除与右除按第 13.3 节规定。

算术运算按第 5.4 节的数值提升规则确定结果类型。整数 `/` 产生精确 fraction；`~/`、`%` 接受整数与可精确转换的数值。除数为零时产生 `MathError`；位移计数为负数时产生 `MathError`。`^` 按优先级表结合；矩阵幂按第 13.3 节规定。

6.6 比较与逻辑

比较运算返回布尔值。链式比较由 `<`、`<=`、`>`、`>=`、`==`、`!=`、`in`、`is`、`<:` 组成：`a < b < c` 等价于 `(a < b) && (b < c)`，其中 `b` 只求值一次。`<=>` 单独使用；`<=>` 与其他比较运算符连续出现且缺少括号时产生 `SyntaxError`。`&&` 与 `||` 返回布尔值，操作数按真值解释。`!` 返回操作数真值的逻辑补值。三元表达式 `c ? a : b` 在 `c` 为真时求值 `a`，否则求值 `b`。

比较按第 5.5 节的相等、同一性与排序规则求值。`in` 对容器执行成员测试，对类型执行值成员测试，对实例调用 `__contains__` 或字段回退。`<:` 对集合执行子集测试，对类型执行类型子集测试，对 class 执行继承或 trait 组合测试。`is` 只比较对象 identity。NaN 与其他值的相等结果为假；NaN 参与 `<=>` 时产生 `TypeError`。

`&&` 在左侧为假时返回 `false`，`||` 在左侧为真时返回 `true`。其余情况求值右侧，并按真值返回布尔结果。

6.7 调用

调用表达式 `f(a, b, c = v, ...rest)` 的语义为：

1. 求值 `f`；
2. 按第 8 节的参数绑定规则建立参数；
3. 执行函数体；
4. 返回函数体结果，或传播其错误。

方法调用 `a.f(args)` 将 `a` 作为接收者绑定到 `self`；接收者只占用 `self` 通道。调用期间建立新的调用帧；调用结束恢复调用者的作用域与 defer 栈。调用产生错误时，该帧的 defer 执行，pending ref 状态清理，错误传播到调用者。部分应用闭包在调用时补齐 hole；实参数量覆盖记录的 hole 数量。

6.8 赋值

简单赋值 `x = e` 先求值 `e`，再更新 `x` 的绑定。复合赋值 `x op= e` 等价于读取 `x`、求值 `e`、执行 `op`、写回 `x`，其中 `x` 的目标表达式只求值一次。属性目标 `a.f = e` 与索引目标 `a[i] = e` 分别调用属性写入与索引写入；目标对象与索引表达式各求值一次。解构赋值按第 9.3 节执行 staging 与 commit。

6.9 索引与切片

`x[i]` 按以下规则求值：

- list、string、matrix 按索引读取；
- dict 按键读取；
- 实现了 `__getitem__` 的实例按该方法读取。

负索引从末尾回绕。越界读取产生 `ValueError`。`x[i:j:k]` 产生切片；切片边界按第 13.2 节规定。切片起点与终点按长度回绕；步长为零时产生 `ValueError`。`slice(start, end, step)` 构造一级 slice 值，字段 `start`、`end`、`step` 可读。`x[s]` 等价于 `x[s.start:s.end:s.step]`；省略的边界按容器长度处理。索引赋值的可写性由值类型决定；只读值上的索引赋值产生 `RuntimeError`。dict 键缺失读取产生 `RuntimeError`；dict 键赋值新增或更新字段。list 索引赋值更新已有位置；越界赋值产生 `ValueError`。matrix 索引赋值产生 `RuntimeError`。

7 控制表达式

7.1 表达式的值

控制形式与声明形式都是表达式。每个表达式产生一个值，或执行一次控制转移。表达式的值按以下规则确定：

形式	值
字面量、标识符、调用	对应值
块	最后一个表达式的值；空块为 `none`
变量绑定、赋值	被绑定或被写入的值
函数绑定	函数值
class、trait、namespace、enum	对应 class 对象、namespace 对象或类型值
import	模块或命名空间值
include	`none`
if	选中分支的值
while、for、for-in	`none`
match	选中分支的值
try	try 体或选中 catch 分支的值
defer	`none`
delete	`none`
macro 调用	展开后表达式的值
return、break、continue、throw	控制转移

7.2 块

块 `{ e1; e2; ... }` 建立新作用域。块的值是最后一个表达式的值；空块的值为 `none`。

块退出时按以下顺序完成清理：保存块结果，执行 defer（后进先出），释放块级绑定。`return`、`break`、`continue` 与错误传播都触发同一清理流程。

7.3 条件与循环

`if (c) e1 else e2` 按 `c` 的真值选择分支，整个表达式的值为选中分支的值。条件求值一次；`else if` 链按顺序求值每个条件，首个为真的分支胜出。`while (c) e` 在每次迭代前求值 `c`；循环正常结束时值为 `none`。`for (init; cond; step) e` 依次执行初始化、条件、循环体与步进；正常结束时值为 `none`。循环体进入下一轮前执行 defer；`break` 结束最近循环，`continue` 进入下一轮。循环体中的 `return` 结束当前函数，并按块清理规则执行 defer。

分支体与循环体可以是由单个表达式组成的隐式块；隐式块中的 `local` 绑定保持局部。`return` 遇到行尾结束，返回值为 `none`；显式返回值写在 `return` 后。逻辑函数形式 `not(x)`、`and(a,b)`、`or(a,b)` 按函数调用求值；`and` 与 `or` 对两个实参求值。`bool(x)` 将值转换为布尔真值。

`break` 结束最近的外层循环；`continue` 进入最近的外层循环的下一轮。`return e`

结束当前函数并返回值 `e`。

7.4 for-in

`for (p in e) body` 对 `e` 的迭代值依次匹配模式 `p` 并执行 `body`。list 按存储顺序迭代；dict 按键插入顺序迭代；set 按插入顺序迭代；string 按 code point 迭代；matrix 按列迭代；实现 `__iter__`/`__next__` 的实例按迭代协议产生值。循环变量默认提升到函数或 namespace 作用域；`local` 修饰保持块级。for-in 按迭代协议求值：取得迭代器，重复调用 `__next__` 并把结果与模式匹配；`StopIteration` 结束循环，模式匹配失败产生 `MatchError`。每轮迭代体退出时执行该轮 defer。迭代器由 list、dict、set、string、matrix 或实现 `__iter__`/`__next__` 的实例提供。for-in 模式支持默认值、rest、const 与逐目标修饰符：

```
for (@[local key, local val] in dict("a", 1)) { print(key) }
for (@[const k, v] in data) { ... }
```

7.5 match

`match (e) { p1 => e1, p2 => e2 }` 依次尝试分支。分支模式匹配成功时执行对应表达式并返回其值。模式变量在分支内建立局部绑定。模式匹配失败时尝试下一分支；全部分支失败时产生 `MatchError`。guard 在模式绑定后求值；guard 为假时该分支失败，匹配继续。分支体的 defer 在分支退出时执行。

7.6 defer

`defer e` 将表达式 `e` 注册到当前块。块以任何方式结束时，注册的 defer 按后进先出顺序执行。`return` 先求值返回值，再执行 defer。正常退出时 defer 产生错误，该错误替代正常结果并传播。已有错误时 defer 产生错误，主错误保留，次生错误进入 `suppressed` 列表。

7.7 try、catch 与 throw

`try { ... } catch (...) { ... }` 是表达式。try 体正常结束时，整个表达式返回 try 体的值。try 体产生错误时，按 catch 分支顺序匹配：

- 模式匹配成功时执行该分支并返回其值；
- 模式匹配失败时尝试下一分支；
- 全部分支失败时重新抛出原错误。

`catch` 支持解构、类型匹配、or-pattern 与 guard。`try { ... }` 与 catch 子句并肩出现；catch 子句省略时，表达式吸收错误并返回 `none`。

catch 匹配按以下顺序执行：

1. 求值 try 体；
2. try 体产生错误时，按分支顺序尝试模式；
3. 模式匹配成功后建立分支绑定并求值 guard；
4. guard 为真时执行分支体，整个表达式返回分支体值；
5. 模式或 guard 失败时尝试下一分支；
6. 全部失败时重新抛出原错误。

catch 分支退出时执行该分支的 defer。catch 体内 `throw e` 重新抛出错误；`return`、`break`、`continue` 穿过 try/catch 边界并执行相应清理。

`throw e` 将 `e` 包装为 `Exception` 并传播。`error("msg")` 与 `throw "msg"` 等价。

8 函数与参数

8.1 闭包

函数与 lambda 在定义位置捕获词法环境。函数值是闭包；闭包包含参数列表、函数体与捕获环境。自由名字在捕获环境中解析。局部函数捕获自身名字的引用，因此可以递归调用自身。

函数绑定可以带 `const`、`local`、`ref`、`state` 修饰符。`const f(x) = ...` 建立只读函数绑定；`local f` 建立块级函数绑定。`ref f` 更新外层函数绑定；`state f` 建立闭包持久函数绑定。函数定义可以省略参数初始化器的类型注解；参数解构、默认值与 rest 规则同时适用。

8.1.1 参数绑定算法

调用按以下顺序建立参数环境：

1. 求值被调用表达式，得到函数值；
2. 按从左到右的顺序绑定位置实参；
3. 按关键字名绑定关键字实参；
4. 关键字实参匹配仅关键字参数；
5. 使用默认值的参数按声明顺序求值默认值；

6. rest 参数收集剩余位置实参；

7. kwargs 参数收集剩余关键字实参；

8. 检查必填参数、重复参数与形参表之外的关键字。

位置实参、关键字实参与 spread 的求值顺序按第 6.3 节执行。绑定完成后，函数体在新建的调用作用域中求值；调用结束返回函数体值或传播控制转移。

参数位置可以直接使用 dict、list 或矩阵模式：

```
dist({x, y}) = sqrt(x^2 + y^2)
process(@[a, b = 10, ...rest]) = a + b + rest.sum()
get_info(const {name, age = 20}) = f"{name} is {age}"

process(@[5])              // 15
process(@[5, 20, 1, 2])    // 28
```

参数模式的目标按模式规则建立局部绑定；const 模式参数保持只读。参数模式支持默认值、rest 与嵌套模式。

8.2 默认参数

参数默认值在调用时求值，按参数声明顺序，每个缺失参数求值一次。提供实参时跳过对应默认值。默认值可以读取外层变量与前面已绑定的参数。

8.3 ref 与 const 参数

`ref` 参数绑定到调用处的 lvalue。实参为字面量或临时值时产生 `TypeError`；实参为 const 变量时产生 `RuntimeError`。`ref` 参数带默认值时产生 `SyntaxError`。`const ref` 参数绑定到只读引用；对该参数的写入在编译期产生 `SyntaxError`。`const` 参数的写入在编译期产生 `SyntaxError`。

8.4 rest、kwargs 与 spread

`...rest` 收集剩余位置参数为 list。`...kw` 收集形参表之外的关键字参数为 dict。调用中的 `...expr` 展开位置实参；； `...expr` 展开关键字实参。展开值必须为可迭代对象或映射对象；传入值属于其他类型时产生 `TypeError`。`args.apply(f)` 将 list 或矩阵展开为位置实参，等价于 `f(...args)`。内置函数可以被同作用域的用户函数或变量遮蔽；`delete` 删除遮蔽绑定，内置函数继续可用。

```
f(a, b = 2) = a + b
f(1)            // 3
f(1, b = 3)     // 4
```

```
g(a, ...rest; b, ...kw) = @[a, rest, b, kw]
g(1, 2, 3, b = 4, x = 5)
// @[1, @[2, 3], 4, {"x": 5}]
```

8.5 占位符与部分应用

_ 作为调用实参时产生部分应用闭包。部分应用闭包记录已提供的实参与 hole 数量。调用部分应用闭包时，一次调用提供全部 hole；hole 数量少于记录值时产生 `RuntimeError`。

```
add(a, b, c) = a + b + c
p = add(1, _, 3)
p(2)            // 6
q = add(_, _, _)
q(1, 2, 3)      // 6
```

9 模式与解构

9.1 模式种类

模式包括：

- 通配符 _；
- 变量模式；
- 字面量模式；
- list、set、dict、矩阵模式；
- 字段模式；
- 类型模式。

9.2 匹配

匹配产生绑定环境。变量模式绑定整个值；容器模式递归匹配元素；字段模式按键或字段匹配；类型模式检查值是否属于指定类型。rest 模式匹配剩余元素并绑定为容器。

9.2.1 分支选择与 guard

分支按书写顺序尝试，第一个成功匹配且 guard 为真的分支胜出。guard 绑定到整个分支；or-pattern 中每个备选共享同一 guard。guard 失败时整个分支失败，匹配继续到下一分支。分支变量在分支体内为局部绑定，分支结束后退出作用域。全部模式失败时产生 `MatchError`。

or-pattern 的每个备选绑定相同的变量名集合；备选变量集合出现差异时产生 `SyntaxError`。`x: int, x: string => ...` 是合法 or-pattern，两个备选绑定同一个 `x`。

9.2.2 动态断言与求值顺序

模式按从左到右求值。动态断言模式写作 `(expression)`，求值结果与主题按相等规则比较。先绑定的变量可以在同一模式的后续断言中使用：

```
match (val) {
    [a, (2 * a)] => "second is twice the first",
    {x: a, y: (a ^ 2)} => "y is the square of x",
    (math.pi()) => "pi",
    _ => "other"
}
```

纯标识符模式绑定新变量；字面量模式直接比较常量。

9.2.3 容器模式

list 模式 `@[...]` 匹配 list；矩阵模式 `[...]` 匹配矩阵，逗号分隔列、分号分隔行。矩阵模式按行列匹配：`[x, y]` 匹配一行两列，`[x; y]` 匹配两行一列。尾随分号固定行数，例如 `[a, b;]` 精确匹配一行，剩余行不进入 rest。默认目标允许输入缺少对应列，rest 捕获剩余列或行：

```
[a, b = 10, ...rest] = [5]
// a == 5, b == 10, rest == []
```

rest 模式 `...name` 捕获剩余元素；同一行最多一个 rest。行内 rest 位于分号前时捕获该行剩余列；位于末尾 `]` 前时捕获剩余行。

dict 模式 `{...}` 匹配 dict、实例字段与 namespace 字段。字段可以带 `local`、`ref`、`state`、`const` 修饰符。简写 `{name}` 等价于 `{"name": name}`。`{a: int}` 把字段 `a` 解构到名为 `int` 的变量；类型断言写作 `{a: a: int}`。rest 捕获剩余字段并绑定为 dict，位于字段列表末尾。

9.2.4 视图提取

实例实现 `__match__` 时，匹配使用该方法返回的 list 或 dict 视图。视图返回实例自身时，匹配回退到默认字段视图，保持递归终止。

9.3 解构赋值

解构赋值分两阶段：

1. staging：求值右侧，建立临时值与候选绑定；
2. commit：全部匹配、类型检查与 const/ref 预检成功后写入目标。

任一阶段失败时，目标绑定保持赋值前状态。默认值可读取同一模式中前面已绑定的目标；前向引用产生 `SyntaxError`。

9.4 state 目标

目标级 `state` 在第一次调用时初始化；初始化失败时保持待初始化状态，下一次调用重试。已初始化的 `state` 目标在模式默认值中复用其值。pattern 级 `state` 的右侧表达式按闭包首次调用求值一次，整体提交。

10 声明表达式

10.1 变量声明

变量声明建立绑定，并可按类型注解检查初始值。`local`、`ref`、`state` 省略初始化器时捕获外层绑定；class 数据属性省略初始化器时进入待初始化状态，读取该属性产生 `RuntimeError`。`const` 绑定在初始化后保持只读；再次赋值产生 `RuntimeError`。

10.2 class 与 trait

class 体包含属性；属性可以保存数据值，也可以保存可调用闭包。trait 体包含抽象属性、默认属性、属性默认值与 private 属性。本规范在需要区分声明形态时使用“数据属性”和“方法属性”；对象模型统一称属性（property）。文法中的 `field-decl` 与 `method-decl` 描述声明形式，两者都建立属性绑定。

class 可以继承一个父类，并组合多个 trait。trait 可以组合多个 trait。trait 定义求值为 class 对象，并带 trait 标记；`type(T) == class_type`，`istrait(T)` 为真。trait 对象用于组合与一致性判定，实例化入口由 class 提供。

class、trait、namespace、enum 的名称形式包括：

```
class Name { ... }          // 名称绑定到 Name
Name = class { ... }        // 匿名类，赋给 Name
P = class "Point" { ... }   // 字符串名称，绑定由赋值建立
local class Temp { ... }    // 局部绑定
```

标识符名称建立同名绑定；字符串名称设置定义名，绑定由外围赋值建立。字符串名称使用字符串字面量；f-string 作为定义名产生 `ParserError`。`local`、`const` 等修饰符需要变量绑定；字符串名称与修饰符组合产生 `SyntaxError`。匿名 class 的 `{` 与 `class` 位于同一行；`extends` 形式同样保持同行。

10.3 属性

属性修饰符：

修饰符	语义
`static`	类级属性，定义时求值
`const`	属性的只读属性
`local`	private 属性，作用域限于声明体
`static local`	private static 属性

数据属性默认值在实例构造时初始化。方法属性绑定闭包值。实例属性省略初始化器时，构造结束前必须由 `init` 赋值；缺失时产生 `RuntimeError`。`const` 属性接受一次赋值；后续写入产生 `RuntimeError`。

`static` 属性属于 class 对象，定义 class 时求值。实例属性查找读取实例属性表；static 属性通过 class 目标读取。静态方法通过 class 目标调用，`self` 绑定该 class 对象。`static local` 属性保持 private，外部读取产生 `RuntimeError`。

`self` 在调用时绑定接收者。方法体中的 `self.name` 访问实例属性；读取缺失属性时沿 class 链解析。实例属性优先于 class 属性。隐式协议调用沿 class 链解析，忽略实例上的同名覆盖。

`init` 在实例创建时自动执行；`finalize` 在实例失去可达性时执行。两个名字由生命周期机制保留；在实例上手动调用产生 `RuntimeError`。

实例支持字符串键索引作为属性访问：`p["x"]` 读取属性 `x`，`p["x"] = v` 写入属性 `x`。类省略 `__getitem__` 时，实例索引使用字符串键访问属性；字符串之外的索引产生 `TypeError`。

10.4 属性查找

显式属性访问 `obj.name` 按以下顺序查找：

1. 实例属性表；
2. 实例所属 class 的属性表，包括组合 trait 与父类链的合并结果；
3. 原生类型的原型属性表。

查找在首个命中处停止。实例属性覆盖 class 属性；子类属性覆盖父类同名属性。`local` 属性只允许在声明它的 class 或 trait 方法内通过 `self.name` 访问；外部代码与子类访问产生 `RuntimeError`。`static` 属性只存在于 class 对象；`C.name` 查找 class 静态属性表，`obj.name` 读取实例属性表。`super.name` 从父类属性表开始查找，用于显式调用父类实现。

隐式协议调用（算术、比较、hash、字符串化、索引、迭代、调用）从 class 链查找协议属性，实例属性表只参与显式属性访问。

10.5 类型一致性

实例对 class 或 trait 的一致性按 class 链与组合 trait 判定：

1. 实例的 class 等于目标 class；
2. 实例的 class 沿父类链到达目标 class；
3. 实例的 class 或祖先 class 组合了目标 trait。

`obj in C`、`C1 <: C2`、`isinstance(obj, C)`、类型注解与 match 类型模式使用同一规则。`class B with A` 的实例属于 A；`B <: A` 为真。trait 的抽象方法由 class 或祖先 class 实现；声明签名与实现签名一致。

10.6 继承与初始化

数据属性默认值按 base 到 derived 的顺序初始化。同一 class 层中，trait 数据属性先于该 class 自身数据属性初始化。class 数据属性默认值可以读取其 trait 数据属性；trait 数据属性默认值读取 trait 自身属性。

全部数据属性初始化完成后执行 `init`。子类省略 `init` 时继承父类构造；子类定义 `init` 时，父类构造由 `super.init(...)` 显式调用。JC2 支持单一父类继承；子类继承父类的全部方法。`extends` 接受表达式，可以引用 module 中的类。`super.method(...)` 分派到父类实现。

`finalize` 在实例失去可达性后执行，至多一次。`init` 产生错误时，已分配实例仍进入 finalize。finalize 产生的错误被隔离，后续 finalize 继续执行。

10.7 宿主类引用

在 class 或 trait 的方法体内，`class` 求值为当前宿主类对象。`class(...)` 以该宿主类构造实例，例如 `make() = class()`。宿主类对象可以参与类型检查、参数传递与返回值。

10.8 trait 组合

多个 trait 的同名属性按组合顺序覆盖：后组合的 trait 覆盖先组合的 trait。class 自身属性覆盖 trait 属性。trait 的 private 属性按各自词法作用域解析。class 及其父类必须实现每个抽象方法，并保持声明签名；实现条件出现缺口时，class 定义产生 `TypeError`。

抽象方法以省略函数体的方法声明表示；实现方法必须匹配参数个数、参数名与返回类型。trait 默认方法提供实现，class 可以覆盖。trait 组合沿 `with` 列表摊平；父 trait 的抽象方法可以由 class 或子 trait 的默认方法满足。

10.9 dunder 协议

实例可以实现以下协议方法：

方法	调用场景
`__add__`、`__sub__`、`__mul__`、`__div__`、`__idiv__`、`__ldiv__`、`__mod__`、`__pow__`	算术
`__lshift__`、`__rshift__`、`__bitand__`、`__bitor__`、`__bitxor__`	位移与位运算
`__radd__`、`__rsub__`、`__rmul__`、`__rdiv__`、`__ridiv__`、`__rldiv__`、`__rmod__`、`__rpow__`、`__rlshift__`、`__rrshift__`、`__rbitand__`、`__rbitor__`、`__rbitxor__`	反射算术与位运算
`__neg__`、`__bitnot__`	一元运算
`__eq__`、`__neq__`、`__lt__`、`__le__`、`__gt__`、`__ge__`	比较
`__cmp__`、`__rcmp__`	三向比较与反射
`__contains__`、`__subsets__`	成员与子集
`__getitem__`、`__setitem__`	索引
`__getattr__`、`__setattr__`	动态字段
`__iter__`、`__next__`	迭代
`__call__`	实例调用
`__match__`	模式匹配视图
`__unpack__`、`__mapping__`	位置/关键字 spread
`__str__`、`__repr__`	字符串转换
`__len__`、`__abs__`、`__bool__`	长度、绝对值、真值
`__hash__`	hash 计算

协议方法沿 class 链查找。`__cmp__` 返回实数，符号决定顺序；返回 bool 或 NaN 时产生 `TypeError`。`__eq__` 与 `__hash__` 同时存在时，相等实例产生相同 hash。`__eq__` 与 `__lt__` 可以推断 `!=`、`<=`、`>`、`>=`；完全序场景使用推断，偏序场景显式实现其余比较方法。

隐式协议调用使用 class 属性，实例上的同名覆盖只影响显式调用。`__match__` 返回 list 或 dict 作为匹配视图；`__unpack__` 返回位置 spread 的 list；`__mapping__` 返回关键字 spread 的 dict。类提供 `__contains__` 时，`x in obj` 使用该方法；类省略 `__contains__` 时，`x in obj` 检查属性与 `__getattr__`。实例属性查找优先于 class 属性；static 属性通过 class 目标访问。

10.10 反射

反射函数读取 class 与实例结构：

函数	结果
`type(x)`	值的类型对象
`getClass(obj)`	实例的 class 定义
`getParent(C)`	父类或 `none`
`getName(x)`	class、实例、trait、namespace 或 enum 的名称
`getFields(obj)`	public 实例属性名列表，含数据与方法属性
`getFields(C)`	class 的实例属性名列表，含继承属性
`getStatics(C)`	class 的 public static 属性名列表，含继承属性
`hasField(obj, name)`	public 属性存在性
`getFieldInfo(obj, name)`	属性描述 dict：`isMethod`、`isStatic`、`isConst`、`isAbstract`；未知属性返回 `none`
`isinstance(obj)`	任意 class 实例
`isinstance(obj, C)`	class 或父类/组合 trait 的实例
`istrait(x)`	trait 定义

属性名列表按名称排序；`local` 属性与 private static 属性由反射隐藏。`getFields(C)` 读取 class 的实例属性表；`getStatics(C)` 读取 static 属性表。`getClass(obj)` 返回实例的 class 定义对象；`getParent(C)` 沿单一父类链返回。

```
class A { x = 1; m() = self.x }
getFields(A())        // @["m", "x"]
getFields(A)          // @["m", "x"]
hasField(A(), "m")    // true
getClass(A()) == A    // true
```

```
getName(A)            // "A"
getParent(A)          // none
```

10.11 enum

enum 定义是表达式，值为冻结 namespace。每个成员形成一个 public 字段，字段名为成员名，字段值为成员值。

成员值的确定规则：

1. 首个成员省略显式值时取 0；
2. 后续成员省略显式值时取前一个整数值加 1；
3. 显式值必须是编译期常量，例如字面量、1 << 0 或常量折叠表达式；
4. 前一个成员值属于整数之外的类型时，后续成员采用显式值；
5. 显式值可以混合类型，例如字符串与整数。

成员名的形式包括标识符与关键字；关键字在 enum 体内按成员名处理。enum Name { ... } 建立同名绑定；enum "Name" { ... } 设置定义名，绑定由外围赋值建立；省略名称时形成匿名 enum，左花括号与 enum 同行。local enum 与 const enum 按变量修饰符规则作用于绑定。

enum 的值为 namespace_type，成员按字段读取：

```
enum Color { Red, Green = 5, Blue }
Color.Red         // 0
Color.Green       // 5
Color.Blue        // 6
type(Color)       // namespace_type
```

enum namespace 固定：成员写入与新增字段产生 RuntimeError。enum 可作为 dict key 或 set 元素。for (name in Color) 依次产生成员名；for (@[name, value] in Color) 产生成员名与成员值。getFields(Color) 列出成员名。

11 namespace 与模块

11.1 namespace

namespace 是词法作用域。顶层变量中的 public 绑定在 namespace 构造结束时导出为 public 字段。导出值为该变量在构造结束时的值。local 绑定的作用域终止于 namespace 体。具名 namespace 在体内建立同名 const 自绑定；对该名字赋值在编译期产生 SyntaxError。

namespace 函数是普通闭包。函数体中的自由名字按词法环境解析。`self` 在调用时绑定到调用目标；`self.name` 访问该目标的字段。

namespace 是动态字段表：成员可以在运行时新增、修改与删除。`namespace` 在 namespace 体内求值为该 namespace 自身；`class` 在 class 方法内求值为宿主类。`freeze(ns)` 锁定字段表，此后写入产生 `RuntimeError`；冻结 namespace 可哈希。namespace 支持 `keys()`、`values()` 与 for-in 迭代；成员缺失时按字段表结果处理。namespace 体是普通执行块，关键字可以作为变量名的成员由外部赋值建立。

```
namespace N {
    x = 1
    f() = self.x
}
N.x = 2
N.f()          // 2

A = namespace { a = 3 }
B = namespace { include A; g() = self.a }
B.g()          // 3
```

11.2 include

`include expr` 将 namespace 的 public 字段复制到当前 namespace。目标由求值时的动态执行栈确定：最内层正在构造的 namespace（或模块）接收复制。函数体内的 `include` 使用调用时最内层正在构造的 namespace；执行栈中没有正在构造的目标时产生 `RuntimeError`。复制规则：

- public 字段进入目标；`local` 字段保持在源 namespace；
- 标量字段形成独立快照；对象字段共享引用；
- `const` 属性随字段复制；
- 函数按值复制，保留源闭包捕获；调用时 `self` 绑定目标 namespace；
- 直接字段与 include 字段同名时产生 `RuntimeError`；复制保持原子性。

  动态 include 字段属于运行期字段，访问路径为 `self.name` 或 `Target.name`。

11.3 导入

导入形式包括：

```
import module          // 绑定 module
```

```
import pkg.sub          // 绑定 pkg
import "path"           // 返回 namespace 值
import @module          // 编译期导入，建立编译期绑定
```

`import module` 等价于 `module = import "module"`。`const import module`、`local import module` 按变量修饰符作用于导入绑定。`import @module` 在编译期导入宏与 syntax 定义；运行时绑定表保持为空。

字符串导入按以下顺序解析：

1. 精确路径，或路径加 `.jc2`、`.jcb`、`.dll`；
2. 包含 `init.jc2` 的目录 package；
3. `lib/` 或当前目录中的原生库；
4. `.jc2` 源文件，然后 `.jcb` 字节码，然后目录 package。

标识符导入优先选择原生库。同一库或文件的重复导入返回同一对象。

package 目录以 `init.jc2` 为入口；子模块通过显式 `import pkg.table` 加载。`import pkg` 加载入口并绑定根名；`import pkg.table` 同时加载入口与子模块，访问路径为 `pkg.table`。同名源文件与原生库、同名文件与目录 package 产生歧义错误。package namespace 缺少成员时产生错误。模块内的 `local` 绑定保持私有；普通定义导出为 namespace 字段。模块/库缺失时产生 `RuntimeError`；package 循环产生 `RuntimeError`。导入过程产生错误时，导入中止；后续导入重新执行。已导入的 namespace 是共享单例；冻结共享单例会影响后续导入者。私有冻结副本由 `copy(import "lib", true)` 建立。

`include` 对同一字段的重复复制保持幂等；字段冲突按本节的 include 规则产生错误。

```
m = import "lib.jc2"
m.x
m2 = import "lib.jc2"
m is m2        // true

const local lib = import "lib.jc2"
```

12 宏与 syntax

12.1 macro

`macro` 在编译期执行。参数为 AST 节点，返回 AST 节点或 AST 节点列表。宏定义体为 `function-body`：单个表达式或一个块。宏调用形式为 `@name(...)`、`@name(...) { ... }` 或 `@name { ... }`。尾随块作为最后一个参数。可变参数收集剩余 AST 节点。

`quote { ... }` 产生 Block 节点；`quote expr` 产生单个表达式节点。`$` 在 quote 内插入 AST 节点。返回列表时，节点按顺序平铺到调用位置。

宏定义在编译期作用域中；内层定义遮蔽外层定义，离开作用域后恢复外层定义。`ast.gensym(prefix)` 产生新的变量节点，保持生成名字独立。

宏展开发生在语法分析阶段：解析器读取宏调用，求值宏函数，取得 AST 节点并替换调用位置。宏函数接收调用处的 AST 节点；参数按位置绑定，尾随块作为最后一个参数。宏函数返回单个 AST 节点或 AST 节点列表；列表按顺序平铺，并保持调用位置作用域。宏函数返回其他值时产生 `RuntimeError`；宏名缺失时产生 `ParserError`。递归宏在展开前注册占位定义，因此宏体可以调用自身。引号块中的 `$name` 插入已绑定的 AST 节点；其他值产生展开错误。

```
macro twice(node) = quote {
    $node
    $node
}

macro square(node) = quote { $node * $node }

macro make_swap(a, b) = {
    import ast
    t = ast.gensym("t")
    return quote {
        local $t = $a
        $a = $b
        $b = $t
    }
}
```

```
count = 0
@twice { count += 1 }     // count == 2
@square(5 + 2)            // 49
x = 1
y = 2
@make_swap(x, y)          // x == 2, y == 1
```

12.2 宏卫生

`ast.gensym(prefix)` 返回新的变量节点，名字包含唯一序号。宏应使用 gensym 为临时变量命名，保持名字独立。宏参数与调用处表达式保持原始节点；展开结果的绑定遵循普通作用域规则。

12.3 ASTNode 结构

每个 AST 节点是 `ASTNode` 实例，包含 `type`（字符串）、`line` 与 `position`；其余字段按节点类别保存。本表列出规范使用的核心字段；宏系统暴露的完整字段集合由 `ast` 模块定义。

节点类别	字段
Literal	value、isString、isImaginary、isKeyword
Variable、Unary、Binary	name；op、right；op、left、right
GroupingExpr、SequenceExpr	expression；expressions
Assign、DestructAssign	name/pattern、value、isRef、isState、isLocal、isConst、typeHint
CompoundAssign	target、op、value、isRef、isState、isLocal
Block、IfExpr	statements；condition、thenBranch、elseBranch
WhileExpr、ForExpr	condition、body；initializer、condition、update、body
ForInExpr、MatchExpr	pattern、iterable、body、isLocal、isConst；subject、branches
TryCatchExpr、DeferExpr	tryBody、branches；body
Call、InvokeExpr、MethodCallExpr	callee、arguments；callee；object、method、arguments
LambdaExpr	name、params、paramIsRef、paramIsConst、defaultExprs、restName、paramTypes、returnType
MatrixNode、ListNode、SetLiteral	elements
DictLiteral	entries（键值节点对）
IndexAccess、DotAccess	object、indices；object、field
FStringExpr	literals、exprs、formatSpecs
ClassDefExpr	name、superClassExpr、staticProperties、instanceProperties
NamespaceDecl	name、body
EnumDefExpr、ImportExpr	name、members；path

MacroDefExpr、	name、params、restName、isTokenMacro、body；
MacroCallExpr	macroName、arguments
QuoteExpr、	body；expr
UnquoteExpr	
VariablePattern、	name、modifier、isConst、typeHint
RestPattern	
ListPattern、	elements、rest；rows、restRow
MatrixPattern	
DictPattern、	entries、rest；inner、defaultExpr
DefaultPattern	

`ast.toNode(value, [line], [position])` 从 none、bool、int、BigInt、float 或 string 构造 Literal 节点。`ast.toValue(node)` 将 Literal 节点转换为标量值；其他节点产生 `TypeError`。`ast.gensym(prefix, [line], [position])` 构造唯一名称的 Variable 节点。直接读取或修改节点字段使用普通实例字段规则；节点类型与字段名按上表约定。

12.4 syntax

`syntax` 的输入是 `TokenStream`。

`stream.parse()` 解析剩余 token；`stream.parseOne()` 解析一个节点。Token 的 `type` 与 `lexeme` 同步更新；解析器检查两者一致。syntax 返回 AST 节点或列表，平铺规则与 macro 相同。TokenStream 提供 `tokens`、`peek`、`advance`、`match`、`consume`、`insert`、`set`、`remove` 与 `isAtEnd`；Token 包含 `type`、`lexeme`、`line`、`position`。Token 类型与 lexeme 保持一致；两者出现差异时产生 `TypeError`；token 流缺少闭合时产生 `ParserError`。

```
syntax passthrough(stream) = stream.parse()
x = @passthrough { 1 + 2 }     // 3

syntax plus_to_star(stream) = {
    for (t in stream.tokens()) {
        if (t.type == "PLUS") {
            t.type = "STAR"
            t.lexeme = "*"
        }
    }
    return stream.parse()
}
```

```
@plus_to_star { 2 + 3 }        // 6
```

13 内建值域

本卷规定核心值语义、索引、切片、运算符、哈希与错误边界。

13.1 字符串

字符串是只读值，长度、索引、切片与迭代按 UTF-8 code point 计算。负索引从末尾回绕。切片形式为 `s[i:j:k]`。字符串支持 `+` 拼接与 `*` 重复。字符串索引赋值产生 `RuntimeError`。

`ord(s)` 返回首字符 code point；`chr(n)` 返回对应字符。`ord` 的参数为空字符串时产生 `RuntimeError`。`parseNum(s)` 解析十进制整数、浮点与指数形式。`int(s)` 解析十进制、十六进制、二进制与八进制整数字符串。

格式化字符串的插值形式为 `f"text {expr::spec} text"`。格式说明支持填充、对齐、符号、`#`、零填充、宽度、分组、精度与类型。插值表达式按当前作用域求值。

```
s = "中文a"
len(s)          // 3
s[0]            // "中"
s[-1]           // "a"
s[1:3]          // "文a"
```

13.2 容器

list 按存储顺序迭代。dict 按插入顺序迭代键、值与键值对。set 按插入顺序迭代，集合并、交、差与对称差按集合语义计算。set 相等忽略顺序。

```
L = @[10, 20, 30]
L[-1]           // 30
L[1:3]          // @[20, 30]
L[1:2] = @[9]   // @[10, 9, 30]

d = {"a": 1}
d.b = 2          // {"a": 1, "b": 2}
d["a"]           // 1

s = @{1, 2}
s | @{2, 3}     // @{1, 2, 3}
```

```
@[x * 2 for (x in @[1, 2])]              // @[2, 4]
@{x * 2 for (x in @[1, 2]) if (x > 1)}   // @{4}
{x: x * 2 for (x in @[1, 2])}             // {1: 2, 2: 4}
```

list、string 支持负索引与切片；越界读取产生 `ValueError`。list slice 赋值要求替换长度与选中切片长度一致。dict 缺键赋值新增字段。

dict key 与 set 元素必须是可哈希值。普通 list 可以作为容器元素；冻结副本可以作为 key 或元素。

13.2.1 list

list 是引用值；赋值共享同一实例。索引读写按第 6.9 节执行。list 越界访问产生 `ValueError`。

13.2.2 dict

dict 按键插入顺序迭代。键读取缺失时产生 `RuntimeError`；键赋值新增或更新。

13.2.3 set

set 保持插入顺序，忽略重复元素。集合运算包括并 `|`、交 `&`、差 `-`、对称差 `^^`。集合相等比较元素集合，迭代按插入顺序。

13.2.4 哈希与键

dict key 与 set 元素必须可哈希。可哈希值集合与相等规则由第 5.5 节规定。数值、字符串、slice、矩阵、冻结容器、namespace、enum 与实现 `__hash__` 的实例可哈希；可变 list、dict、set 作为键时产生 `TypeError`。相等值产生相同 hash；hash 在对象生命周期内保持稳定。

13.3 矩阵

矩阵字面量为 `[row; row; ...]`。`matrix(r,c,...)`、`toMatrix(list)` 按行构造。`toList(matrix)` 按行输出。

`A[i]` 返回第 i 列；`A[i,j]` 返回元素。`A[r,:]`、`A[:,c]`、`A[r0:r1,c0:c1]` 返回行、列或子矩阵。`len(A)` 返回元素总数。

矩阵是只读值；索引赋值产生 `RuntimeError`。矩阵迭代按列进行。

`A * B` 为矩阵乘法；`A / B` 为右除 $A \cdot B^{-1}$；`A \ B` 为左除 $A^{-1} \cdot B$；`A ^ n` 为整数幂。矩阵与标量相加 `A + c`、`c + A` 在方阵上对对角线加 c，非方阵产生 `MathError`；矩阵与矩阵相加按同形状逐元素进行。

零元素矩阵规范化为 0×0 矩阵。

矩阵形状条件失败时，加、乘、除与拼接产生 `ValueError` 或 `TypeError`。矩阵使用列主序存储；`toList` 按行输出嵌套 list；迭代按列产生列向量。

```
A = [1, 2; 3, 4]
A[0, 1]        // 2
A[0]           // [1; 3]
A[0:2, 0:2]    // [1, 2; 3, 4]
A + 1          // [2, 2; 3, 5]
A \ [1; 0]     // [-2; 3/2]
```

14 错误

14.1 异常值

`throw e` 产生 `Exception` 实例。该实例包含：

字段	含义
`type`	错误类型名
`message`	错误消息或结构化数据
`traceback`	调用栈文本
`suppressed`	defer 中产生的次生异常列表

`throw "msg"` 产生 `type = "Exception"`、`message = "msg"` 的实例。`error("msg")` 与该形式等价。`type(e)` 返回异常类；`e.type` 返回该类的类型名。

14.2 内置错误类型

内置错误类型继承 `Exception`：

```
TypeError  ValueError  MathError  MatchError  IOError
RuntimeError  OverflowError  CalculusError  SymbolicError
StopIteration
```

`StopIteration` 在迭代协议结束时产生，并由迭代协议消费。

编译期诊断使用 `LexerError`、`ParserError` 与 `SyntaxError`，并携带源文件与行号。这些类型描述词法、语法与静态检查阶段；运行时异常类型为继承 `Exception` 的类实例。

错误类型	典型来源
TypeError	类型条件失败、参数类型检查失败
ValueError	索引越界、取值失败
MathError	除零、模零、数值运算错误
MatchError	模式匹配失败
IOError	文件与流操作失败
RuntimeError	运行时状态错误、只读容器写入
OverflowError	数值范围溢出
CalculusError	微积分运算失败
SymbolicError	符号运算失败
StopIteration	迭代结束

错误类型是 Exception 的子类；catch 的类型模式按 class 链与 trait 组合检查。类型模式匹配失败时该分支跳过；catch 的全部类型分支失败时原错误重新抛出。catch (e) 捕获任意异常；catch ({message: {code, msg}}) 解构错误消息。

```
try {
    1 + "x"
} catch (e) {
    e.type          // "TypeError"
    e.message       // "Cannot add 'int' and 'string'."
    type(e)         // TypeError
}

try {
    throw Exception("HttpError", {code: 500})
} catch (e) {
    e.type          // "HttpError"
    e.message.code  // 500
}
```

14.3 错误传播

错误沿调用栈传播，直到遇到匹配的 catch 分支或到达程序边界。catch 分支按顺序匹配。类型匹配失败时尝试下一分支；全部分支失败时重新抛出原错误。defer 中产生的错误在当前块结束时传播；已有错误时，新错误进入 suppressed 列表。

14.4 assert

`assert(cond)` 与 `assert(cond, msg)` 在 `cond` 为假时产生 `RuntimeError`。`assert(label, got, expected)` 在 `got != expected` 时产生带标签与期望/实际值的 `RuntimeError`。

15 一致性要求

一致性实现必须满足：

1. 词法与文法：接受本规范的全部合式程序；
2. 名字与类型：按本规范解析作用域、执行类型检查；
3. 求值：按本规范规定的顺序与规则产生值；
4. 错误：按本规范规定的错误类型与时机终止或传播；
5. 模块：按本规范规定的导入、namespace 与 include 语义建立绑定；
6. 宏：在编译期按本规范展开；
7. 内建值域：按本规范实现字符串、容器与矩阵的可观察行为。

实现可以对诊断文本、调试信息、内存布局、执行策略作出选择。这些选择保持本规范规定的可观察结果。

A 文法汇总

A.1 词法产生式

```
source-file   = { whitespace | comment | token } EOF ;
token         = identifier | keyword | number | string
              | operator | delimiter | newline | error-token ;
whitespace    = " " | "\t" | "\f" | newline ;
newline       = "\n" | "\r\n" | "\r" ;
comment       = line-comment | block-comment ;
line-comment  = "//" { any-char-except-newline } newline ;
block-comment = "/*" { block-comment | comment-text } "*/" ;
comment-text  = ? any char except "*/" ? ;
identifier    = plain-identifier | quoted-identifier ;
plain-identifier = letter { letter | digit | "_" } ;
quoted-identifier = "`" { any-char-except-backtick-newline } "`" ;
```

```
letter        = "A".."Z" | "a".."z" | "_" | unicode-letter ;
digit         = "0".."9" ;
decimal-integer = digit { [ "_" ] digit } ;
hex-integer   = ( "0x" | "0X" ) hexdigit { [ "_" ] hexdigit } ;
bin-integer   = ( "0b" | "0B" ) bindigit { [ "_" ] bindigit } ;
oct-integer   = ( "0o" | "0O" ) octdigit { [ "_" ] octdigit } ;
float-number  = digit { [ "_" ] digit } "." digit { [ "_" ] digit }
              [ exponent ]
              | digit { [ "_" ] digit } exponent ;
exponent      = ( "e" | "E" ) [ "+" | "-" ] digit { digit } ;
imaginary-number = ( decimal-integer | hex-integer | bin-integer | oct-
   integer | float-number ) "i" ;
quoted-string = '"' { string-char } '"' | "'" { string-char } "'" ;
triple-string = '"""' { any-char } '"""' | "'''" { any-char } "'''" ;
raw-string    = ordinary-raw | raw-triple | delimited-raw ;
ordinary-raw  = 'r"' { any-char-except-quote } '"' | "r'" { any-char-except
   -quote } "'" ;
raw-triple    = 'r"""' { any-char } '"""' | "r'''" { any-char } "'''" ;
delimited-raw = 'r"' [ raw-delimiter ] "(" { any-char } ")" [ raw-delimiter
    ] '"'
              | "r'" [ raw-delimiter ] "(" { any-char } ")" [ raw-delimiter
    ] "'" ;
raw-delimiter = identifier ;
f-string      = "f" ( '"' { any-char } '"' | "'" { any-char } "'"
              | '"""' { any-char } '"""' | "'''" { any-char } "'''" ) ;
number        = decimal-integer | hex-integer | bin-integer
              | oct-integer | float-number | imaginary-number ;
string        = quoted-string | triple-string | raw-string | f-string ;
literal       = number | string | "true" | "false" | "none" ;
operator      = "+" | "-" | "*" | "/" | "~/" | "\\" | "%" | "^"
              | "=" | "==" | "!=" | "<" | "<=" | ">" | ">=" | "<=>"
              | "&&" | "||" | "!" | "~" | "&" | "|" | "^^"
              | "<<" | ">>" | "<:" | "|>" | "+=" | "-=" | "*="
              | "/=" | "%=" | "~/=" | "\\=" | "^=" | "&=" | "|="
              | "^^=" | "<<=" | ">>=" ;
delimiter     = "(" | ")" | "[" | "]" | "{" | "}" | "," | ";" | ":"
              | "." | "?" | "@" | "$" | "..." | "->" | "=>" ;
hexdigit      = digit | "a".."f" | "A".."F" ;
bindigit      = "0" | "1" ;
octdigit      = "0".."7" ;
```

```
any-char      = ? any Unicode code point except EOF ? ;
any-char-except-newline = ? any Unicode code point except newline ? ;
any-char-except-quote = ? any Unicode code point except quote and EOF ? ;
any-char-except-backtick-newline = ? any Unicode code point except backtick
    and newline ? ;
string-char   = ? any Unicode code point except quote, backslash, newline ?
    ;
unicode-letter = ? Unicode letter category ? ;
backslash     = "\\" ;
error-token   = ? invalid lexical unit ? ;


keyword       = "if" | "else" | "while" | "for" | "in" | "is"
              | "as" | "break" | "continue" | "return" | "local"
              | "ref" | "state" | "const" | "static" | "delete"
              | "throw" | "try" | "catch" | "import" | "include"
              | "match" | "macro" | "syntax" | "quote" | "defer"
              | "class" | "trait" | "with" | "extends"
              | "namespace" | "enum" | "super" | "self"
              | "true" | "false" | "none" ;
```

A.2 程序与表达式

```
program        = { expression } EOF ;
block          = "{" { expression } "}" ;

expression     = sequence ;
sequence       = assignment { "," assignment } ;
assignment     = ternary [ assign-op assignment ] ;
assign-op      = "=" | "+=" | "-=" | "*=" | "/=" | "%="
               | "~/=" | "\\=" | "^=" | "&=" | "|=" | "^^="
               | "<<=" | ">>=" ;
ternary        = pipe [ "?" expression ":" ternary ] ;
pipe           = logical-or { "|>" pipe-step } ;
pipe-step      = "." ( identifier | keyword ) [ "(" [ arguments ] ")" ]
               | logical-or ;
logical-or     = logical-and { "||" logical-and } ;
logical-and    = comparison { "&&" comparison } ;
comparison     = bitwise-or { compare-op bitwise-or } ;
```

```
compare-op     = "==" | "!=" | "<" | "<=" | ">" | ">="
               | "<=>" | "in" | "is" | "<:" ;
bitwise-or     = bitwise-xor { "|" bitwise-xor } ;
bitwise-xor    = bitwise-and { "^^" bitwise-and } ;
bitwise-and    = shift { "&" shift } ;
shift          = additive { ( "<<" | ">>" ) additive } ;
additive       = multiplicative { ( "+" | "-" ) multiplicative } ;
multiplicative = unary { ( "*" | "/" | "~/" | "\\" | "%" ) unary } ;
unary          = ( "!" | "-" | "+" | "~" ) unary | power ;
power          = as-expr [ "^" unary ] ;
as-expr        = postfix { "as" postfix } ;
postfix        = primary { call-suffix | index-suffix | member-suffix } ;
call-suffix    = "(" [ arguments ] ")" ;
index-suffix   = "[" index-arg { "," index-arg } "]" ;
index-arg      = expression
               | [ expression ] ":" [ expression ] [ ":" [ expression ] ] ;
member-suffix  = "." ( identifier | keyword ) ;
```

A.3 构造与字面量

```
primary        = literal | identifier | "(" expression ")"
               | lambda | list-literal | set-literal | dict-literal
               | matrix-literal | list-comprehension
               | matrix-comprehension | set-comprehension
               | dict-comprehension
               | quote-expr | macro-call | block
               | binding | function-binding | import-expr | include-expr
               | delete-expr
               | context-self-expr | class-expr | trait-expr
               | namespace-expr | enum-expr | macro-def | syntax-def
               | if-expr | while-expr | for-expr | for-in-expr
               | match-expr | try-expr | defer-expr
               | return-expr | break-expr | continue-expr | throw-expr ;

list-literal   = "@[" [ matrix-row { ";" matrix-row } ] "]" ;
set-literal    = "@{" [ expression { "," expression } ] "}" ;
dict-literal   = "{" [ dict-entry { "," dict-entry } ] "}" ;
dict-entry     = expression ":" expression
               | "..." expression
```

```
               | identifier ;
matrix-literal = "[" [ matrix-row { ";" matrix-row } ] "]" ;
matrix-row     = expression { "," expression } ;
lambda         = "(" [ params ] ")" "=>" ( expression | block ) ;
quote-expr     = "quote" ( block | expression ) ;
macro-call     = "@" identifier [ "(" [ arguments ] ")" ] [ block ] ;
```

A.4 声明与成员

```
binding        = [ "const" ] [ "local" | "ref" | "state" ]
                 binding-target [ ":" expression ] [ "=" expression ] ;
binding-target = identifier | pattern ;
function-binding = [ "const" ] [ "local" | "ref" | "state" ]
                 identifier "(" [ params ] ")" [ "->" expression ]
                 "=" function-body ;
function-body  = expression | block ;

name-spec      = [ "$" ] identifier | string ;
import-expr    = "import" [ "@" ] ( module-path | string ) ;
module-path    = identifier { "." identifier } ;
include-expr   = "include" expression ;
delete-expr    = "delete" [ "@" ] identifier { "," identifier } ;
namespace-expr = "namespace" [ name-spec ] block ;
class-expr     = "class" [ name-spec ] [ "extends" expression ]
                 [ "with" trait-list ] class-body ;
trait-expr     = "trait" [ name-spec ] [ "with" trait-list ] class-body ;
trait-list     = expression { "," expression } ;
class-body     = "{" { class-member } "}" ;
class-member   = { member-modifier } ( field-decl | method-decl ) ;
member-modifier = "static" | "local" | "const" ;
field-decl     = member-name [ "=" expression ] ;
method-decl    = member-name "(" [ params ] ")" [ "->" expression ]
                 [ "=" function-body ] ;
member-name    = identifier | keyword ;
enum-expr      = "enum" [ name-spec ] "{" { enum-member } "}" ;
enum-member    = ( identifier | keyword ) [ "=" expression ] ;
macro-def      = "macro" [ "$" ] identifier "(" [ macro-params ] ")"
                 "=" function-body ;
macro-params   = macro-param { "," macro-param } ;
```

```
macro-param    = [ "..." ] identifier ;
syntax-def     = "syntax" [ "$" ] identifier "(" identifier ")"
                 "=" function-body ;
context-self-expr = "class" | "namespace" ;
```

A.5 控制表达式

```
if-expr        = "if" "(" expression ")" expression
                 [ "else" expression ] ;
while-expr     = "while" "(" expression ")" expression ;
for-expr       = "for" "(" [ expression ] ";" [ expression ] ";"
                 [ expression ] ")" expression ;
for-in-expr    = "for" "(" for-pattern "in" expression ")" expression ;
for-pattern    = [ "const" ] [ "local" ] pattern ;
match-expr     = "match" "(" expression ")" "{" match-arms "}" ;
match-arms     = match-arm { "," match-arm } ;
match-arm      = branch-patterns [ "if" "(" expression ")" ] "=>"
   expression ;
try-expr       = "try" expression [ catch-clause ] ;
catch-clause   = "catch" ( "(" branch-patterns ")" expression
                         | "{" match-arms "}" ) ;
defer-expr     = "defer" expression ;
return-expr    = "return" [ expression ] ;
break-expr     = "break" ;
continue-expr  = "continue" ;
throw-expr     = "throw" expression ;
```

A.6 参数、实参与推导式

```
params         = param { "," param } [ ";" param { "," param } ]
               | ";" param { "," param } ;
param          = [ "const" ] [ "ref" | "local" | "state" ]
                 ( "..." identifier | pattern [ ":" expression ]
                   [ "=" expression ] ) ;
arguments      = argument { "," argument } [ ";" argument { "," argument }
   ]
               | ";" argument { "," argument } ;
argument       = expression | identifier "=" expression | "..." expression
   ;
```

```
comp-clause    = "for" "(" for-pattern "in" expression ")"
                 { "if" "(" expression ")" } ;
comp-clauses   = comp-clause { comp-clause } ;
list-comprehension   = "@[" expression comp-clauses "]" ;
matrix-comprehension = "[" expression comp-clauses "]" ;
set-comprehension    = "@{" expression comp-clauses "}" ;
dict-comprehension   = "{" expression ":" expression comp-clauses "}" ;
```

A.7 模式

```
pattern        = primary-pattern [ "=" expression ] ;
primary-pattern = "_" | variable-pattern | literal
               | list-pattern | matrix-pattern
               | dict-pattern | rest-pattern | dynamic-assert-pattern
               | [ "local" | "ref" | "state" | "const" ] primary-pattern ;
variable-pattern = identifier [ ":" expression ] ;
rest-pattern   = "..." identifier [ ":" expression ] ;
list-pattern   = "@[" [ pattern { "," pattern } ] "]" ;
matrix-pattern = "[" [ pattern-row { ";" pattern-row } [ ";" ] ] "]" ;
pattern-row    = pattern { "," pattern } ;

dict-pattern   = "{" [ field-pattern { "," field-pattern }
                 [ "," rest-pattern ] ] "}" ;
field-pattern  = [ "const" ] [ "local" | "ref" | "state" ] identifier
                 [ ":" pattern | "=" expression ]
               | string ":" pattern ;
dynamic-assert-pattern = "(" expression ")" ;
branch-patterns = pattern { "," pattern } ;
```

B 一致性矩阵

一致性测试套件至少覆盖以下行为类别。每一行给出规范章节与必须验证的可观察结果。

类别	规范章节	必须验证的行为

源文本	第 2 节	UTF-8 解码、BOM、换行、注释、字面量、指令与词法错误
文法	第 3 节	全部产生式接受、静态约束错误、歧义消解结果
名字与作用域	第 4 节	绑定、重声明、遮蔽、提升、`ref`、`state`、`const`
类型与值	第 5 节	类型对象、联合类型、类型注解、`as` 断言、真值与哈希
表达式求值	第 6 节	求值顺序、短路、优先级、赋值目标、索引与切片
控制流	第 7 节	分支值、循环值、`break`/`continue`、defer、try/catch
函数	第 8 节	默认值、关键字参数、rest/kwargs、spread、ref 参数、partial
模式	第 9 节	分支顺序、guard、or-pattern、矩阵 rest、staging/commit
class 与 trait	第 10 节	成员查找、static、trait 一致性、dunder、init/finalize、enum
namespace 与模块	第 11 节	字段导出、include、导入解析、package、去重与错误
宏与 syntax	第 12 节	展开顺序、quote/unquote、列表平铺、gensym、Token 校验
内建值域	第 13 节	字符串、list/dict/set、矩阵的方法与错误
错误	第 14 节	异常字段、类型匹配、传播、suppressed、assert

实现可以对诊断文本、资源上限与内部布局作出选择；表中行为必须按规范结果产生。一致性报告应列出每类行为的测试入口与结果。

C 规范用语索引

用语	出现位置
绑定	第 4 节

作用域	第 4 节
提升	第 4 节
真值	第 5.3 节
提升（数值）	第 5.4 节
求值顺序	第 6.3 节
错误	第 14 节
一致性	第 15 节